

**TẬP ĐOÀN CÔNG NGHIỆP – VIỆN THÔNG QUÂN ĐỘI**

## **BÁO CÁO MINI-PROJECT**

### **SOC AI Search**

Natural Language Search for Security Events

|                             |                             |
|-----------------------------|-----------------------------|
| <b>Sinh viên thực hiện:</b> | <Tên sinh viên thực hiện>   |
| <b>Email:</b>               | <Email sinh viên thực hiện> |
| <b>Chương trình:</b>        | Viettel Digital Talent 2026 |
| <b>Lĩnh vực:</b>            | Software Engineer           |
| <b>Mentor:</b>              | <Tên mentor>                |
| <b>Đơn vị:</b>              | <Tên đơn vị>                |

**Hà Nội, 2026**

# Lời mở đầu

Trong các trung tâm vận hành an toàn thông tin (Security Operations Center – SOC), khối lượng log và cảnh báo cần phân tích tăng nhanh theo quy mô hệ thống. Analyst phải thường xuyên trả lời các câu hỏi như “có bao nhiêu đăng nhập thất bại từ một quốc gia trong 24 giờ qua”, “top IP nào tạo nhiều cảnh báo nhất”, hoặc “xu hướng failed login theo giờ như thế nào”. Với các hệ thống tìm kiếm như Elasticsearch, các câu hỏi này thường phải được chuyển thành Query DSL hoặc aggregation DSL. Việc viết DSL thủ công không chỉ mất thời gian mà còn dễ sai, đặc biệt với người mới hoặc trong các tình huống cần phản ứng nhanh.

Đề tài SOC AI Search được xây dựng để giải quyết vấn đề trên theo hướng sử dụng mô hình ngôn ngữ lớn (Large Language Model – LLM) như một trợ lý tạo kế hoạch truy vấn, nhưng không trao quyền thực thi trực tiếp cho AI. Người dùng đặt câu hỏi bằng tiếng Anh hoặc tiếng Việt; LLM chỉ được phép sinh ra một cấu trúc trung gian có kiểm soát gọi là SearchPlan; backend sau đó parse, validate, compile thành Elasticsearch DSL và thực thi. Cách tiếp cận này giữ được sự tiện lợi của natural language search nhưng vẫn đảm bảo guardrail về bảo mật, kiểm toán và khả năng giải thích.

Báo cáo này trình bày bối cảnh, mục tiêu, kiến trúc, phương pháp triển khai, kết quả đạt được và các hướng phát triển tiếp theo của hệ thống. Nội dung tập trung vào các phần cốt lõi: pipeline Natural Language → SearchPlan → DSL, guardrail AI, search/aggregation, audit/history, CSV export, RBAC bằng Keycloak, giao diện SOC dashboard và triển khai thực tế trên DigitalOcean với Caddy HTTPS.

# Tóm tắt nội dung và đóng góp

Mini-project SOC AI Search là một hệ thống hỗ trợ SOC analyst tìm kiếm, thống kê và điều tra log bảo mật bằng ngôn ngữ tự nhiên. Hệ thống gồm frontend React/TypeScript, backend Spring Boot, Elasticsearch làm event store, PostgreSQL làm audit/history store, Keycloak làm identity provider, cùng LLM provider dạng mock hoặc Gemini. Sản phẩm đã được triển khai public trên DigitalOcean VPS, định tuyến domain bằng Name.com và HTTPS tự động qua Caddy.

Các đóng góp chính của đề tài gồm:

- Thiết kế contract SearchPlan làm lớp trung gian an toàn giữa câu hỏi tự nhiên và Elasticsearch DSL.
- Xây dựng pipeline parse/validate/compile để ngăn LLM sinh DSL tùy ý hoặc truy vấn trực tiếp Elasticsearch.
- Hỗ trợ search raw events và aggregation cơ bản gồm count, group\_by, top\_n, date\_histogram.
- Cung cấp Query Transparency: frontend hiển thị SearchPlan đã validate và DSL đã compile để người dùng kiểm tra.
- Tích hợp audit/history, pin investigation, replay query và export CSV an toàn theo query\_id.
- Tích hợp Keycloak OIDC và RBAC với ba vai trò SOC\_VIEWER, SOC\_ANALYST, SOC\_ADMIN.
- Xây dựng UI dark theme cho SOC console, dashboard, raw events, aggregation charts và investigation workflow.
- Thiết lập CI/CD với GitHub Actions, Docker Compose, smoke test và deploy lên VPS.

# Mục lục

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Lời mở đầu</b>                                       | <b>i</b>  |
| <b>Tóm tắt nội dung và đóng góp</b>                     | <b>ii</b> |
| <b>1 Giới thiệu</b>                                     | <b>1</b>  |
| 1.1 Bối cảnh thực tế . . . . .                          | 1         |
| 1.2 Mục tiêu đề tài . . . . .                           | 1         |
| 1.3 Phạm vi triển khai . . . . .                        | 2         |
| 1.4 Đóng góp của mini-project . . . . .                 | 2         |
| <b>2 Nội dung và phương pháp</b>                        | <b>3</b>  |
| 2.1 Tổng quan kiến trúc hệ thống . . . . .              | 3         |
| 2.2 Công nghệ sử dụng . . . . .                         | 3         |
| 2.3 Mô hình dữ liệu sự kiện . . . . .                   | 4         |
| 2.4 Pipeline Natural Language Search . . . . .          | 5         |
| 2.5 Tích hợp LLM: Mock và Gemini . . . . .              | 7         |
| 2.6 SearchPlan: contract trung gian an toàn . . . . .   | 7         |
| 2.7 Validation và guardrails . . . . .                  | 8         |
| 2.8 Compiler và Elasticsearch DSL . . . . .             | 9         |
| 2.9 Search và Aggregation response . . . . .            | 9         |
| 2.10 AI summary best-effort . . . . .                   | 10        |
| 2.11 Audit, history và investigation workflow . . . . . | 10        |
| 2.12 CSV export an toàn . . . . .                       | 11        |
| 2.13 Xác thực và phân quyền . . . . .                   | 11        |
| 2.14 Frontend SOC Console . . . . .                     | 12        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 2.15     | SOC Overview Dashboard và query suggestions . . . . .        | 13        |
| 2.16     | Triển khai và CI/CD . . . . .                                | 13        |
| <b>3</b> | <b>Kết quả thực hiện và đánh giá</b>                         | <b>15</b> |
| 3.1      | Kết quả chức năng . . . . .                                  | 15        |
| 3.2      | Kịch bản demo . . . . .                                      | 16        |
| 3.3      | Đánh giá về an toàn AI . . . . .                             | 17        |
| 3.4      | Đánh giá hiệu năng và trải nghiệm . . . . .                  | 17        |
| 3.5      | Đánh giá bảo mật và RBAC . . . . .                           | 18        |
| 3.6      | Kiểm thử . . . . .                                           | 18        |
| 3.7      | Triển khai thực tế . . . . .                                 | 19        |
| 3.8      | Hạn chế . . . . .                                            | 19        |
| <b>4</b> | <b>Kết luận</b>                                              | <b>21</b> |
|          | Hướng phát triển tương lai . . . . .                         | 21        |
| <b>A</b> | <b>Phụ lục: API tiêu biểu</b>                                | <b>23</b> |
| A.1      | Natural Language Search . . . . .                            | 23        |
| A.2      | Technical SearchPlan . . . . .                               | 23        |
| <b>B</b> | <b>Phụ lục: Câu hỏi phản biện thường gặp</b>                 | <b>25</b> |
| B.1      | Nếu AI sinh sai thì sao? . . . . .                           | 25        |
| B.2      | Tại sao không cho LLM sinh Elasticsearch DSL luôn? . . . . . | 25        |
| B.3      | CSV export có an toàn không? . . . . .                       | 25        |
| B.4      | Dashboard có gọi LLM không? . . . . .                        | 25        |

# Danh mục hình vẽ

|     |                                                                 |    |
|-----|-----------------------------------------------------------------|----|
| 2.1 | Kiến trúc tổng quan của SOC AI Search . . . . .                 | 3  |
| 2.2 | Pipeline Natural Language Search . . . . .                      | 6  |
| 2.3 | Tách biệt LLM provider khỏi parser/validator/compiler . . . . . | 7  |
| 2.4 | CSV export theo cơ chế replay SearchPlan . . . . .              | 11 |
| 2.5 | Minh họa bố cục chính của giao diện Event Search . . . . .      | 13 |
| 2.6 | CI/CD pipeline triển khai lên VPS . . . . .                     | 14 |

# Danh mục bảng biểu

|     |                                                      |    |
|-----|------------------------------------------------------|----|
| 2.1 | Technology stack của hệ thống . . . . .              | 4  |
| 2.2 | Schema chính của SOC event . . . . .                 | 5  |
| 2.3 | Một số rule validation quan trọng . . . . .          | 8  |
| 2.4 | Mapping aggregation sang Elasticsearch DSL . . . . . | 9  |
| 2.5 | RBAC matrix của hệ thống . . . . .                   | 12 |
| 3.1 | Tổng hợp kết quả chức năng đã triển khai . . . . .   | 16 |
| 3.2 | Nhóm kiểm thử tiêu biểu . . . . .                    | 19 |

# Chương 1

## Giới thiệu

### 1.1 Bối cảnh thực tế

Trong môi trường doanh nghiệp, hệ thống thông tin tạo ra lượng lớn log từ nhiều nguồn như Windows authentication, VPN, firewall, endpoint, application gateway và cloud services. SOC analyst cần phát hiện các hành vi bất thường như brute force, account lockout, malware detected, privilege escalation hoặc lưu lượng từ quốc gia rủi ro. Những tác vụ này thường yêu cầu truy vấn theo thời gian, mức độ nghiêm trọng, người dùng, IP, host, loại sự kiện và nội dung message.

Elasticsearch là một công cụ mạnh cho bài toán tìm kiếm log, nhưng Query DSL của Elasticsearch tương đối dài và có nhiều chi tiết kỹ thuật. Ví dụ, để tìm failed login trong 24 giờ qua từ Trung Quốc, analyst cần biết cách viết `bool.filter`, `range`, `terms`, `track_total_hits`, phân trang và sort. Khi chuyển sang thống kê, analyst còn phải nắm thêm `terms aggregation` hoặc `date_histogram`. Đây là rào cản lớn cho tốc độ điều tra và khả năng onboarding thành viên mới.

LLM có thể giúp chuyển câu hỏi tự nhiên thành truy vấn. Tuy nhiên, nếu cho LLM sinh Elasticsearch DSL trực tiếp rồi execute ngay, hệ thống sẽ đối mặt với nhiều rủi ro: sinh field không tồn tại, query quá lớn, script/wildcard/query string không kiểm soát, hoặc bypass cơ chế audit/RBAC. Vì vậy, đề tài chọn hướng “AI hỗ trợ, backend kiểm soát”.

### 1.2 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng một MVP cho phép SOC analyst hỏi bằng ngôn ngữ tự nhiên, nhận kết quả search/aggregation minh bạch, đồng thời đảm bảo an toàn bằng validator, compiler, RBAC và audit.

Các mục tiêu cụ thể:



1. Cho phép người dùng đặt câu hỏi bằng tiếng Anh hoặc tiếng Việt.
2. Chuyển câu hỏi thành SearchPlan JSON thay vì DSL trực tiếp.
3. Validate chặt chẽ SearchPlan: mode, filter, aggregation type, allowlist field, pagination và bucket limit.
4. Compile SearchPlan thành Elasticsearch DSL bằng backend.
5. Hiển thị kết quả raw events hoặc aggregation charts trên giao diện SOC.
6. Lưu query history/audit vào PostgreSQL để replay, pin, export và truy vết.
7. Bảo vệ các chức năng bằng Keycloak OIDC và RBAC.
8. Triển khai hệ thống chạy thật trên VPS với domain và HTTPS.

### 1.3 Phạm vi triển khai

Phạm vi MVP tập trung vào dữ liệu synthetic SOC events với schema cố định. Elasticsearch lưu toàn bộ event documents, trong khi PostgreSQL chỉ lưu metadata như audit, history, SearchPlan, DSL, summary và thông tin export replay.

Hệ thống hiện hỗ trợ hai chế độ truy vấn:

- **Search mode:** trả danh sách raw events có phân trang.
- **Aggregation mode:** trả kết quả thống kê chuẩn hóa để frontend render chart.

Các tính năng nằm ngoài phạm vi MVP gồm multi-turn chat, vector search, anomaly detection production-grade, multi-tenant enterprise, Kubernetes và streaming realtime quy mô lớn.

### 1.4 Đóng góp của mini-project

Đóng góp quan trọng nhất của đề tài không chỉ là gọi LLM để tạo query, mà là thiết kế một pipeline có guardrail:

**Natural Language → SearchPlan → Validator → Compiler → Elasticsearch**

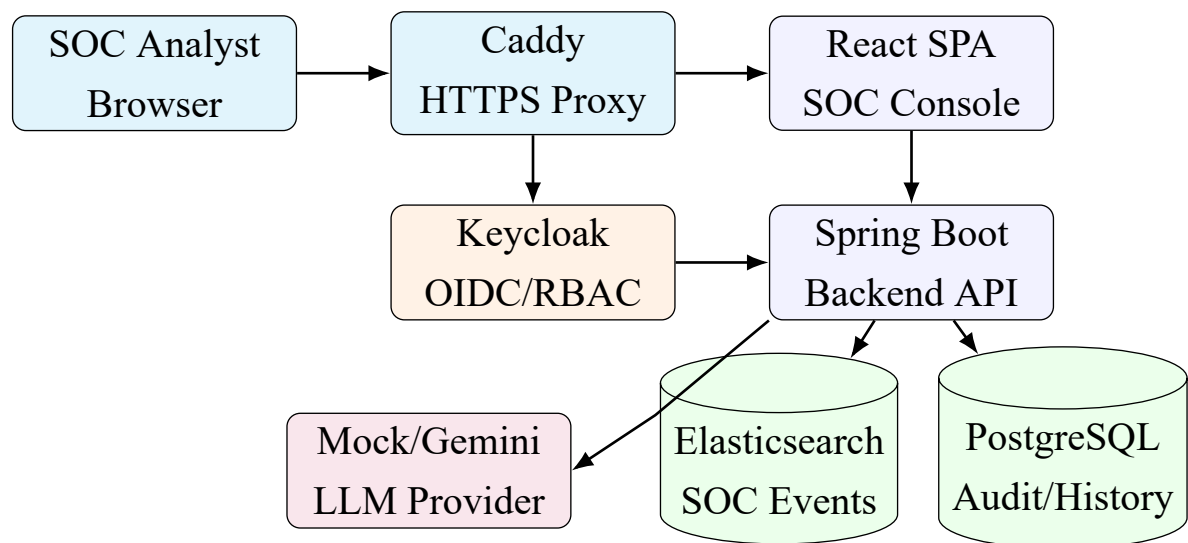
LLM không được truy vấn dữ liệu trực tiếp. Frontend không được gửi DSL tùy ý. Người dùng có thể xem SearchPlan và DSL để hiểu hệ thống đã diễn dịch câu hỏi như thế nào. Đây là yếu tố quan trọng với một hệ thống SOC, nơi tính minh bạch và truy vết thường quan trọng không kém tính tiện dụng.

## Chương 2

# Nội dung và phương pháp

### 2.1 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình modular monolith ở backend. Backend Spring Boot chứa các module search, LLM, validation, compiler, execution, summary, audit, CSV export, event detail và security. Frontend là React SPA độc lập. Các thành phần hạ tầng gồm Elasticsearch, PostgreSQL, Keycloak và Caddy.



**Hình 2.1:** Kiến trúc tổng quan của SOC AI Search

Trong kiến trúc này, frontend không kết nối trực tiếp đến Elasticsearch, PostgreSQL hoặc LLM. Tất cả request nghiệp vụ đi qua backend. Điều này giúp tập trung enforcement tại một nơi: JWT/RBAC, validation, compiler, audit và error handling.

### 2.2 Công nghệ sử dụng

**Bảng 2.1:** Technology stack của hệ thống

| Nhóm              | Công nghệ và vai trò                                                                                                                                                             |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend          | React, TypeScript, Vite, Tailwind CSS, shadcn/ui, lucide-react, Recharts. Dùng để xây dựng SOC console, dashboard, bảng raw events, biểu đồ aggregation và trang investigations. |
| Backend           | Java 21, Spring Boot 3, Spring Security, Spring Web, Bean Validation, Jackson, springdoc-openapi. Dùng để triển khai API, pipeline xử lý SearchPlan, RBAC và Swagger.            |
| Search engine     | Elasticsearch 9.4.2. Lưu SOC events, thực thi query, filter, full-text search và aggregation.                                                                                    |
| Database metadata | PostgreSQL 17. Lưu audit logs, query history, SearchPlan, generated DSL, summary và dữ liệu phục vụ replay/export.                                                               |
| Authentication    | Keycloak. Cung cấp OIDC, JWT, realm roles và user onboarding.                                                                                                                    |
| LLM               | Mock provider cho local/test/CI và Google Gemini cho demo AI thật.                                                                                                               |
| Deployment        | Docker Compose, Caddy, DigitalOcean VPS, Name.com DNS.                                                                                                                           |
| CI/CD             | GitHub Actions, backend tests, frontend tests/build, compose validation, SSH deploy và smoke test domain.                                                                        |

## 2.3 Mô hình dữ liệu sự kiện

Elasticsearch index `soc-events-v1` lưu các event bảo mật. Schema MVP được giữ cố định để compiler có thể sinh DSL an toàn và validator có thể allowlist field.

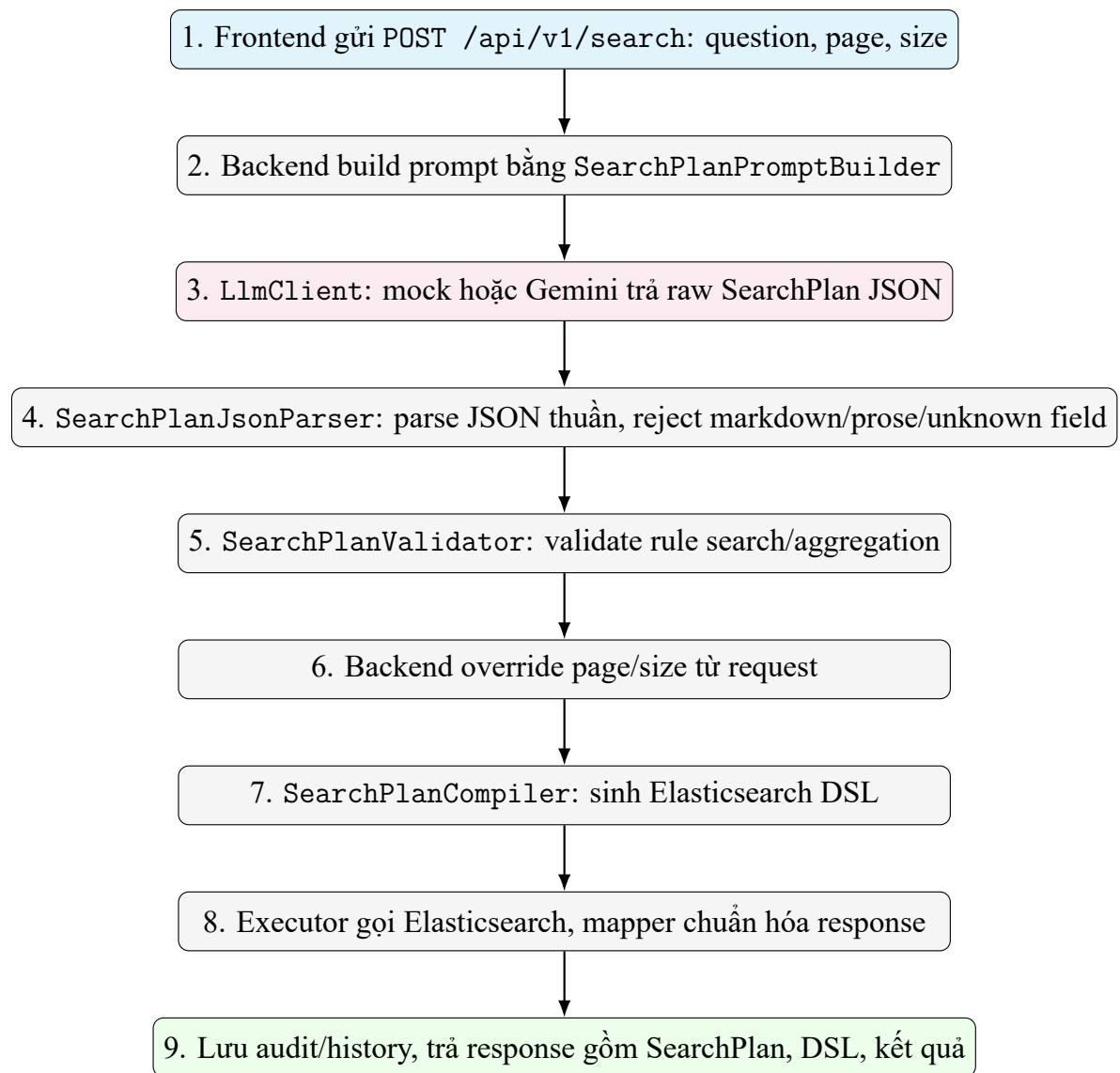
**Bảng 2.2:** Schema chính của SOC event

| Field        | Kiểu trong Elastic-search | Ý nghĩa                                                             |
|--------------|---------------------------|---------------------------------------------------------------------|
| timestamp    | date                      | Thời điểm xảy ra event.                                             |
| source       | keyword                   | Nguồn log như windows-auth, vpn, firewall.                          |
| severity     | keyword                   | Mức độ: low, medium, high, critical.                                |
| event_type   | keyword                   | Loại sự kiện: failed_login, malware_detected, account_lockout, v.v. |
| user         | keyword                   | Tài khoản liên quan.                                                |
| host         | keyword                   | Host hoặc thiết bị liên quan.                                       |
| ip           | ip                        | IP nguồn/đích trong event.                                          |
| country_code | keyword                   | Mã quốc gia như CN, US, VN.                                         |
| message      | text                      | Nội dung mô tả event, hỗ trợ full-text search.                      |
| raw          | text, index=false         | Raw log phục vụ forensic detail.                                    |

Dữ liệu demo được seed bằng PowerShell scripts. PostgreSQL không lưu raw events; nguyên tắc này giúp tách bạch event store và metadata store.

## 2.4 Pipeline Natural Language Search

Pipeline Natural Language Search là phần cốt lõi của đề tài. Người dùng nhập câu hỏi, backend gọi LLM để sinh SearchPlan, sau đó backend kiểm soát toàn bộ phần còn lại.



**Hình 2.2:** Pipeline Natural Language Search

Ví dụ một SearchPlan hợp lệ:

**Mã nguồn 2.1:** Ví dụ SearchPlan cho câu hỏi failed login từ China

```
{
  "mode": "search",
  "filters": {
    "timestamp": {
      "from": "now-24h",
      "to": "now"
    },
    "event_type": ["failed_login"],
    "country_code": ["CN"]
  },
  "page": 0,
```

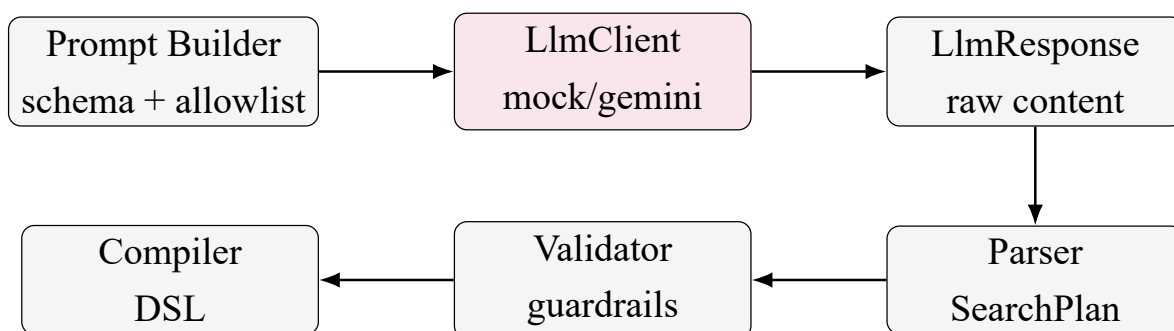
```
"size": 10
}
```

## 2.5 Tích hợp LLM: Mock và Gemini

Hệ thống không tự train model. Backend định nghĩa interface `LlmClient` để trừu tượng hóa provider. Hiện có hai provider:

- **Mock**: trả về `SearchPlan` deterministic cho các câu demo/regression test. Provider này không cần API key, không gọi mạng và phù hợp cho local development, CI, smoke test.
- **Gemini**: gọi Google Gemini REST API bằng cấu hình runtime như provider, base URL, API key và model. Các biến cấu hình tương ứng là `LLM_PROVIDER`, `LLM_BASE_URL`, `LLM_API_KEY` và `LLM_MODEL`.

Gemini client chỉ chịu trách nhiệm gọi API và trích text từ response. Việc parse `SearchPlan`, validate rule và compile DSL hoàn toàn nằm trong backend service khác. Đây là điểm quan trọng để tránh trộn logic AI với logic thực thi.



Hình 2.3: Tách biệt LLM provider khỏi parser/validator/compiler

## 2.6 SearchPlan: contract trung gian an toàn

`SearchPlan` là biểu diễn nghiệp vụ có cấu trúc, dễ hiểu hơn DSL nhưng đủ chặt để backend validate. Contract này gồm:

- `mode`: search hoặc aggregation.
- `filters`: time range, severity, event type, user, host, ip, country code, message query.
- `page, size`: pagination contract.

- **aggregation**: optional, dùng khi mode là aggregation.

SearchPlan không chứa Elasticsearch-specific DSL field như query, aggs, script, wildcard. Nếu LLM hoặc user thêm các field này, parser/validator sẽ reject trước khi compiler sinh DSL.

## 2.7 Validation và guardrails

Guardrail của hệ thống được đặt ở nhiều lớp:

1. **Prompt constraint**: yêu cầu LLM chỉ trả JSON SearchPlan thuần, không markdown, không prose, không DSL.
2. **Jackson strict parser**: reject unknown fields và output không phải JSON object.
3. **Bean Validation**: validate field cơ bản như question not blank, page/size range.
4. **SearchPlanValidator**: validate rule nghiệp vụ: mode, aggregation type, allowlist field, top\_n, interval, message\_query trong aggregation.
5. **Compiler ownership**: chỉ backend compiler được sinh DSL.
6. **RBAC**: backend kiểm tra quyền thao tác như edit SearchPlan, export CSV, xem audit logs.

**Bảng 2.3:** Một số rule validation quan trọng

| Rule                                  | Mục đích                                                                                       |
|---------------------------------------|------------------------------------------------------------------------------------------------|
| Reject unknown fields                 | Ngăn LLM/user thêm field ngoài schema.                                                         |
| Backend override                      | Ngăn LLM tự tăng size hoặc đổi page ngoài request.                                             |
| Aggregation field allowlist           | Chỉ cho phép source, severity, event_type, user, host, ip, country_code.                       |
| Reject .keyword                       | Mapping MVP đã định nghĩa keyword trực tiếp; không để LLM tự gán field Elasticsearch-specific. |
| Reject script, wildcard, query_string | Giảm nguy cơ query nguy hiểm hoặc tốn tài nguyên.                                              |
| Repair tối đa một lần                 | Có khả năng tự sửa lỗi nhẹ nhưng tránh retry loop vô hạn.                                      |

## 2.8 Compiler và Elasticsearch DSL

SearchPlanCompiler nhận một SearchPlan đã validate và sinh Java structure tương thích với Elasticsearch request. Compiler không build JSON bằng nối chuỗi thủ công mà dùng cấu trúc Map<String, Object> hoặc object nội bộ.

Với search mode, compiler sinh bool.filter gồm range, terms, term và match phù hợp. Mặc định sort theo timestamp desc. Với aggregation mode, compiler sinh DSL theo loại aggregation:

**Bảng 2.4:** Mapping aggregation sang Elasticsearch DSL

| Aggregation    | DSL chính                     | Ghi chú                                                |
|----------------|-------------------------------|--------------------------------------------------------|
| count          | size = 0, không có aggs       | Lấy hits.total.                                        |
| group_by       | terms aggregation             | Nếu thiếu top_n, default bucket limit = 20.            |
| top_n          | terms aggregation             | Bắt buộc top_n từ 1 đến 100.                           |
| date_histogram | date_histogram trên timestamp | Dùng fixed_interval: minute = 1m, hour = 1h, day = 1d. |

Generated DSL được trả về response dưới dạng object/map, không phải JSON string escaped. Điều này giúp frontend render pretty JSON trong Query Transparency panel.

## 2.9 Search và Aggregation response

Search response trả về raw events, pagination, summary và telemetry:

**Mã nguồn 2.2:** Schema rút gọn của natural language search response

```
{
  "query_id": "uuid",
  "original_question": "...",
  "mode": "search",
  "search_plan": {},
  "generated_dsl": {},
  "summary": "...",
  "summary_source": "llm",
  "total": 178,
  "page": 0,
  "size": 10,
  "events": []
}
```



Aggregation response không trả event list thật; events là mảng rỗng để frontend xử lý nhất quán. Kết quả chính nằm ở aggregation\_results và chart\_metadata.

**Mã nguồn 2.3:** Schema rút gọn của aggregation response

```
{
  "mode": "aggregation",
  "aggregation_type": "top_n",
  "aggregation_results": [
    { "key": "203.0.113.45", "value": 124 }
  ],
  "chart_metadata": {
    "chart_type": "BAR",
    "x_axis_label": "ip",
    "y_axis_label": "count"
  },
  "events": []
}
```

## 2.10 AI summary best-effort

AI summary giúp analyst đọc nhanh kết quả. Tuy nhiên, summary không được làm hỏng request search. Vì vậy hệ thống áp dụng nguyên tắc best-effort:

- Search result/aggregation result luôn được ưu tiên trả về.
- Nếu LLM timeout hoặc trả summary không hợp lệ, backend dùng deterministic fallback.
- Payload gửi cho LLM được giới hạn: không gửi raw log đầy đủ, không gửi toàn bộ event documents.
- Với aggregation mode, summary dùng trực tiếp aggregation\_results; không query Elasticsearch thêm lần nữa.

Nguyên tắc này giúp trải nghiệm người dùng ổn định hơn, đồng thời tránh chi phí và rủi ro gửi dữ liệu nhạy cảm ra ngoài.

## 2.11 Audit, history và investigation workflow

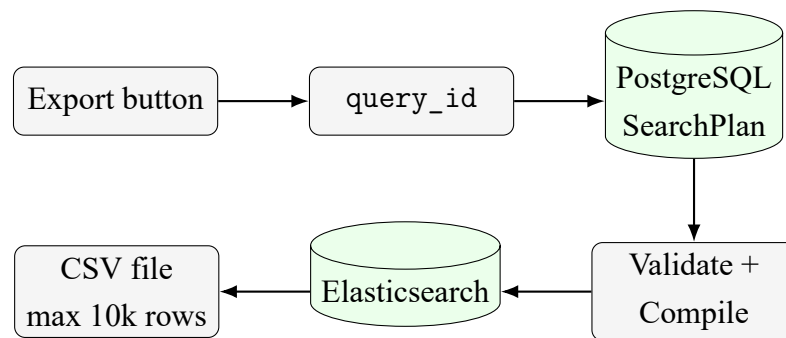
Mỗi truy vấn được lưu vào PostgreSQL trong bảng search\_query\_logs. Bảng này phục vụ ba mục tiêu:

1. **Audit**: biết ai đã query gì, lúc nào, mode nào, status ra sao.
2. **History**: hiển thị Recent Queries và All Investigations trên UI.
3. **Replay**: dùng query\_id để chạy lại query hoặc export CSV.

Các trường chính gồm query id, identity, question, mode, status, failure stage, SearchPlan, generated DSL, result count, latency, summary, error message và created time. Việc lưu SearchPlan/DSL giúp analyst có thể truy vết câu hỏi tự nhiên đã được chuyển thành logic kỹ thuật như thế nào.

## 2.12 CSV export an toàn

CSV export không nhận DSL tùy ý từ client. Thay vào đó, frontend chỉ gửi query\_id. Backend load SearchPlan đã lưu từ PostgreSQL, kiểm tra quyền truy cập, validate lại, compile lại DSL và replay trên Elasticsearch. Kết quả export có giới hạn tối đa 10.000 dòng.



Hình 2.4: CSV export theo cơ chế replay SearchPlan

Cách làm này giảm nguy cơ client gửi DSL độc hại và đảm bảo export vẫn đi qua guardrail backend.

## 2.13 Xác thực và phân quyền

Hệ thống sử dụng Keycloak làm Identity Provider theo chuẩn OIDC. Frontend dùng OIDC flow để lấy access token, sau đó gọi backend bằng Bearer token. Backend là nơi enforce quyền thật sự bằng Spring Security Resource Server.

**Bảng 2.5:** RBAC matrix của hệ thống

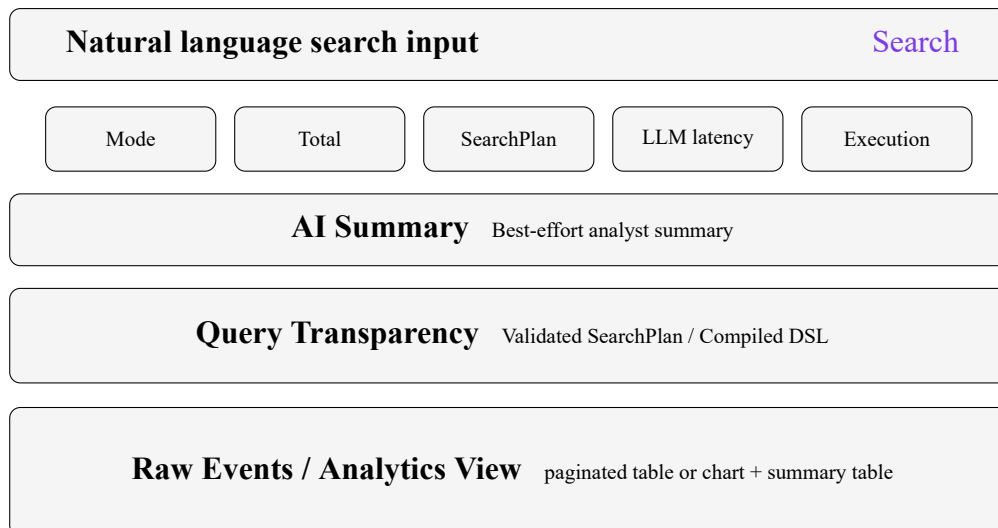
| Chức năng                    | SOC_VIEWER | SOC_ANALYST | SOC_ADMIN |
|------------------------------|------------|-------------|-----------|
| Đăng nhập và xem dashboard   | Có         | Có          | Có        |
| Natural language search      | Có         | Có          | Có        |
| Xem event detail             | Có         | Có          | Có        |
| Edit SearchPlan              | Không      | Có          | Có        |
| Pin investigation            | Không      | Có          | Có        |
| Export CSV                   | Không      | Có          | Có        |
| Xem audit logs toàn hệ thống | Không      | Không       | Có        |

Frontend ẩn/disable các nút không phù hợp để cải thiện UX, nhưng backend vẫn là nguồn kiểm tra quyền cuối cùng. Điều này tránh trường hợp người dùng sửa UI hoặc gọi API thủ công để bypass quyền.

## 2.14 Frontend SOC Console

Frontend được xây dựng theo dark SOC/SIEM style, tập trung vào khả năng quan sát nhanh và minh bạch truy vấn. Các thành phần chính:

- Landing/Auth gateway.
- Event Search với suggested queries và query input.
- KPI cards: mode, total events, SearchPlan status, LLM latency, execution latency.
- AI Summary block.
- Query Transparency: SearchPlan và compiled DSL.
- Raw Events table, Event Detail Drawer.
- Aggregation charts bằng Recharts và summary table.
- SOC Overview Dashboard auto-refresh 10 phút.
- Recent Queries drawer và All Investigations page.
- Admin Audit Logs page.
- Trang All Investigations và Audit Logs áp dụng bố cục **Master-Detail (chia đôi 35/65)** với thanh cuộn độc lập, giúp điều tra log chi tiết mà không làm mất bối cảnh (context) của danh sách tổng.



Hình 2.5: Minh họa bố cục chính của giao diện Event Search

## 2.15 SOC Overview Dashboard và query suggestions

SOC Overview Dashboard dùng các SearchPlan/aggregation cố định thay vì gọi LLM. Các card tối thiểu gồm Total Events, Failed Logins, Critical Alerts, Severity Distribution, Top Source IPs và Events Over Time. Dashboard auto-refresh mỗi 10 phút để cập nhật dữ liệu mới.

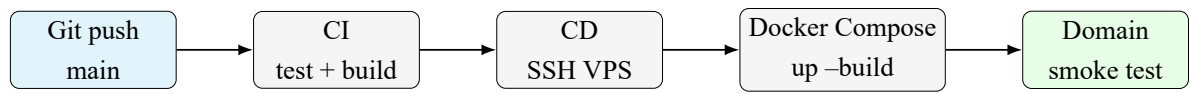
Query suggestions và investigation playbooks tĩnh giúp analyst bắt đầu nhanh với các tình huống như failed login, critical alerts, top source IPs, events by hour. Đây là lựa chọn phù hợp với MVP vì không tốn token LLM, ổn định khi demo và vẫn có giá trị UX.

## 2.16 Triển khai và CI/CD

Hệ thống được deploy trên DigitalOcean Droplet. Domain được mua và quản lý DNS tại Name.com. Caddy làm reverse proxy và tự động cấp HTTPS cho các domain:

- <https://soc-ai-search.app>: frontend.
- <https://api.soc-ai-search.app>: backend API và Swagger.
- <https://auth.soc-ai-search.app>: Keycloak.

Docker Compose chạy các service chính: frontend, backend, Elasticsearch, PostgreSQL, Keycloak và Caddy. GitHub Actions đảm nhiệm CI/CD: test, build, kiểm tra compose config, SSH vào VPS, pull code, rebuild container và chạy smoke test public domain.



**Hình 2.6:** CI/CD pipeline triển khai lên VPS

## **Chương 3**

# **Kết quả thực hiện và đánh giá**

### **3.1 Kết quả chức năng**

Hệ thống đã hoàn thành các nhóm chức năng chính của MVP:

**Bảng 3.1:** Tổng hợp kết quả chức năng đã triển khai

| Nhóm chức năng          | Kết quả                                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Natural Language Search | Hỗ trợ câu hỏi tiếng Anh/Việt qua POST <code>/api/v1/search</code> ; provider mock và Gemini; repair một lần nếu output lỗi. |
| SearchPlan endpoint     | POST <code>/api/v1/search/plan</code> cho phép test validator/compiler/executor không cần LLM.                               |
| Search filters          | Hỗ trợ time range, severity, event type, user, host, ip, country code, message query.                                        |
| Aggregation             | Hỗ trợ count, group by, top N, date histogram; map chart metadata Number/Bar/Line.                                           |
| Query Transparency      | Response và UI hiển thị search_plan và generated_dsl dưới dạng JSON object.                                                  |
| Event detail            | GET <code>/api/v1/events/{event_id}</code> lookup trực tiếp theo Elasticsearch_id.                                           |
| Summary                 | AI summary best-effort với fallback deterministic.                                                                           |
| Audit/History           | Lưu query log vào PostgreSQL, hiển thị Recent Queries/All Investigations, pin/unpin.                                         |
| CSV Export              | Export theo query_id, replay SearchPlan, giới hạn 10.000 rows.                                                               |
| RBAC                    | Keycloak OIDC, role viewer/analyst/admin, backend enforce quyền.                                                             |
| Frontend                | SOC console dark theme, dashboard, charts, raw table, drawer, investigations, audit logs.                                    |
| Deployment              | Public deployment bằng DigitalOcean, Caddy HTTPS, Name.com DNS, GitHub Actions CD.                                           |

## 3.2 Kịch bản demo

Một luồng demo tiêu biểu:

1. Đăng nhập bằng tài khoản analyst.
2. Chạy câu hỏi: *Show me failed login attempts from China in the last 24h.*
3. Mở Query Transparency để xem SearchPlan và compiled DSL.
4. Chạy aggregation: *Show the top 10 source IPs with the most alerts in the last 30 days.*
5. Chạy line chart: *Show failed login trend by hour in the last 24 hours.*
6. Mở SOC Overview Dashboard.

7. Mở All Investigations, pin query, xem SearchPlan/DSL detail.
8. Export CSV bằng tài khoản analyst; kiểm tra viewer không được export.
9. Nếu còn thời gian, đăng nhập admin để xem audit logs.

Kịch bản này thể hiện đầy đủ câu chuyện: hỏi tự nhiên, backend kiểm soát, kết quả minh bạch, audit/replay/export và RBAC.

### 3.3 Đánh giá về an toàn AI

Một câu hỏi quan trọng trong đề án là: “Nếu AI sinh sai thì sao?”. Hệ thống xử lý bằng nhiều lớp:

- LLM không execute query.
- LLM không được sinh DSL.
- Parser reject output không đúng JSON object.
- Validator reject field/rule không hợp lệ.
- Compiler là nơi duy nhất sinh DSL.
- Nếu parse/validation lỗi, repair tối đa một lần.
- Nếu vẫn lỗi, API trả lỗi có kiểm soát, không lộ stack trace.

Cách thiết kế này làm giảm rủi ro so với mô hình “LLM sinh DSL trực tiếp”. Đồng thời, SearchPlan dễ đọc hơn DSL nên analyst có thể kiểm tra và chỉnh sửa trong cơ chế human-in-the-loop, nhưng DSL vẫn read-only.

### 3.4 Đánh giá hiệu năng và trải nghiệm

Trong MVP, trọng tâm không phải benchmark quy mô production mà là trải nghiệm investigation. Một số quyết định thiết kế hỗ trợ hiệu năng:

- Mock LLM dùng cho test/CI để tránh phụ thuộc mạng và quota.
- Dashboard không gọi LLM; dùng aggregation cố định.
- Aggregation bucket limit bị giới hạn bằng top\_n hoặc default 20.



- Search response phân trang, size tối đa 100.
- CSV export giới hạn 10.000 rows, không dùng scroll/search-after phức tạp trong MVP.
- Pagination tiếp theo có thể tái sử dụng SearchPlan đã có, không cần gọi lại LLM.

Frontend hiển thị latency gồm LLM latency, search/execution latency và total latency. Điều này giúp demo rõ phần nào chậm: LLM hay Elasticsearch.

## 3.5 Đánh giá bảo mật và RBAC

Hệ thống có các lớp bảo mật chính:

- Keycloak OIDC để xác thực người dùng.
- JWT resource server ở backend để xác minh token.
- Role hierarchy: admin kế thừa analyst, analyst kế thừa viewer.
- Backend enforce quyền cho export, edit SearchPlan, audit logs.
- CORS cấu hình theo domain frontend.
- Public deployment chỉ expose 80/443/22; backend, ES, PostgreSQL, Keycloak container ports không mở trực tiếp ra internet.
- Không commit API key thật hoặc secret vào repository.

Một điểm quan trọng: UI chỉ đóng vai trò UX gating. Nếu user tự gọi API thủ công, backend vẫn kiểm tra RBAC.

## 3.6 Kiểm thử

Backend có nhiều nhóm test: unit test cho validator/compiler/parser, controller/service test cho endpoint, test RBAC, CSV export, summary, audit, event detail và LLM provider. Bên cạnh đó, hệ thống sử dụng **Swagger UI (springdoc-openapi)** được tích hợp luồng Security Bearer Token, cho phép kiểm thử độc lập các API phân quyền mà không cần đến giao diện Frontend. Frontend dùng Vitest và React Testing Library cho component/service tests. Ngoài ra còn có smoke test PowerShell cho các ngày triển khai và domain smoke test sau deploy.

**Bảng 3.2:** Nhóm kiểm thử tiêu biểu

| Nhóm test                | Mục tiêu                                                                    |
|--------------------------|-----------------------------------------------------------------------------|
| Parser/Prompt            | Đảm bảo output LLM là JSON SearchPlan hợp lệ, repair prompt đúng.           |
| Validator                | Đảm bảo reject field/rule sai, aggregation guardrail đúng.                  |
| Compiler                 | Đảm bảo DSL shape đúng, không sinh script/wildcard/query_string.            |
| Executor/Mapper          | Đảm bảo response từ Elasticsearch được map đúng.                            |
| Natural Language Service | Đảm bảo route search/aggregation, latency, repair, error handling.          |
| RBAC                     | Đảm bảo role viewer/analyst/admin đúng quyền.                               |
| CSV Export               | Đảm bảo export theo query_id, replay SearchPlan, limit rows.                |
| Frontend                 | Đảm bảo render state, permissions, history, export, search API integration. |
| Smoke test               | Đảm bảo local/deploy endpoint hoạt động và domain HTTPS/CORS ổn.            |

### 3.7 Triển khai thực tế

Hệ thống đã được triển khai public trên DigitalOcean Droplet. Caddy đảm nhiệm HTTPS tự động và route theo domain. Docker Compose quản lý service. GitHub Actions thực hiện deploy tự động khi push lên main.

Lợi ích của cách triển khai này:

- Đơn giản, phù hợp MVP và thời gian triển khai ngắn.
- Không cần vận hành Kubernetes, Jenkins hoặc ArgoCD.
- Caddy giảm công sức cấu hình chứng chỉ so với Nginx/Certbot thủ công.
- VPS giúp demo toàn bộ stack thật: frontend, backend, ES, PostgreSQL, Keycloak.

### 3.8 Hạn chế

MVP vẫn có một số hạn chế:

- Dataset là synthetic, chưa tích hợp pipeline log production từ SIEM thật.

- Chưa có multi-turn conversational investigation.
- Aggregation mới ở mức cơ bản, chưa hỗ trợ percentile, cardinality, multi-level group by hoặc pivot table.
- Chưa có saved dashboard động theo người dùng.
- Chưa có observability production-grade như Prometheus/Grafana.
- Chưa có multi-tenant data isolation cho nhiều tổ chức.

Các hạn chế này được xác định rõ để giữ MVP tập trung vào giá trị cốt lõi: natural language search có guardrail, auditability và deploy được.

# Chương 4

## Kết luận

Đề tài SOC AI Search đã xây dựng một MVP hoàn chỉnh cho bài toán hỗ trợ SOC analyst tìm kiếm và phân tích log bảo mật bằng ngôn ngữ tự nhiên. Hệ thống không chỉ dừng ở việc gọi LLM, mà quan trọng hơn là thiết kế một pipeline an toàn: LLM chỉ sinh SearchPlan, backend validate và compile DSL, Elasticsearch chỉ nhận truy vấn do backend tạo ra.

Kết quả đạt được gồm natural language search, aggregation, SearchPlan transparency, AI summary best-effort, audit/history, CSV export, RBAC bằng Keycloak, frontend SOC console, dashboard và triển khai public bằng DigitalOcean/Caddy/GitHub Actions. Các thành phần này chứng minh hệ thống có thể chạy thật, demo thật và có khả năng mở rộng theo hướng enterprise.

Về mặt kỹ thuật, bài học lớn nhất của đề tài là: khi đưa AI vào hệ thống an toàn thông tin, không nên để AI nắm quyền thực thi trực tiếp. Cần có contract trung gian, validator, compiler, RBAC, audit và cơ chế lỗi có kiểm soát. Cách tiếp cận này giúp kết hợp tính tiện dụng của LLM với yêu cầu an toàn và minh bạch của SOC.

## Hướng phát triển tương lai

Các hướng phát triển tiếp theo:

- Multi-turn investigation: giữ ngữ cảnh hội thoại theo session.
- Advanced aggregations: percentile, cardinality, multi-level group by.
- Saved dashboards và saved playbooks theo team SOC.
- Realtime/near-realtime dashboard với auto-refresh linh hoạt hoặc streaming.
- Tích hợp nguồn log thật như Syslog, Beats, Kafka hoặc SIEM connector.
- Báo cáo incident tự động dạng PDF.

- Observability production-grade bằng Prometheus/Grafana.

# Appendix A

## Phụ lục: API tiêu biểu

### A.1 Natural Language Search

Mã nguồn A.1: Request POST /api/v1/search

```
{
  "question": "Show me failed login attempts from China in the last 24h",
  "page": 0,
  "size": 10
}
```

### A.2 Technical SearchPlan

Mã nguồn A.2: Request POST /api/v1/search/plan cho aggregation

```
{
  "mode": "aggregation",
  "filters": {
    "timestamp": {
      "from": "now-24h",
      "to": "now"
    },
    "event_type": ["failed_login"]
  },
  "page": 0,
  "size": 10,
  "aggregation": {
    "type": "date_histogram",
    "interval": "hour"
  }
}
```

```
}  
}
```

# Appendix B

## Phụ lục: Câu hỏi phản biện thường gặp

### B.1 Nếu AI sinh sai thì sao?

LLM không được execute query trực tiếp. Output của LLM phải là SearchPlan JSON. Backend parse, reject unknown fields, validate enum/range/aggregation rules, sau đó compiler mới sinh DSL. Nếu LLM sai, request bị reject hoặc repair tối đa một lần.

### B.2 Tại sao không cho LLM sinh Elasticsearch DSL luôn?

DSL có thể chứa field/query nguy hiểm hoặc ngoài kiểm soát. SearchPlan là contract trung gian giúp hệ thống allowlist field, kiểm soát aggregation, pagination, RBAC và audit.

### B.3 CSV export có an toàn không?

Export chỉ nhận query\_id, sau đó backend replay SearchPlan đã lưu. Client không gửi DSL tùy ý. Export có giới hạn 10.000 rows và vẫn đi qua validate/compile.

### B.4 Dashboard có gọi LLM không?

Không. Dashboard dùng aggregation API cố định, chạy nhanh và ổn định. LLM chỉ dùng cho natural language search và summary best-effort.



# Bibliography

- [1] Elastic, *Elasticsearch Guide: Query DSL and Aggregations*.  
<https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl.html>
- [2] VMware, *Spring Boot Reference Documentation*.  
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [3] VMware, *Spring Security Reference: OAuth2 Resource Server*.  
<https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html>
- [4] Keycloak, *Server Administration Guide*.  
<https://www.keycloak.org/documentation>
- [5] Google AI, *Gemini API Documentation*.  
<https://ai.google.dev/gemini-api/docs>
- [6] Caddy, *Caddy Documentation*.  
<https://caddyserver.com/docs/>
- [7] OWASP, *Application Security Verification Standard and Secure Coding Practices*.  
<https://owasp.org/>